

Panduan Dosen Lab 10 - Monitoring, log, dan SLO

Repo materi: [SeedFlora/meet10CloudService](#) · **durasi contoh:** 100 menit · **jalur utama:** Python standard library. Untuk kasus tiga layanan, hidupkan [Lab 06](#), [Lab 08](#), dan [Lab 09](#) pada satu komputer/Codespace. Checker mandiri tetap dapat berjalan tanpa ketiga layanan.

Hasil belajar dan alasan kasus

Mahasiswa mengoperasikan synthetic monitor, membaca JSONL, menghitung availability dan p95, membuat halaman status, menyimulasikan gangguan, serta menghubungkan metric dengan log request. Kasus kerja: tim operasi melihat web tidak tersedia, tetapi API dan model sentimen masih hidup. Mereka perlu mengisolasi komponen dan menjelaskan sisa error budget. Tekankan bahwa lima probe dalam hitungan detik bukan dasar klaim SLO produksi.

Persiapan dosen

1. Dari root repo ini cek `python --version` dan isi `targets.local.json`: Next `/api/health` 3000, API `/health` 8000, Gradio / 7860. Ketiga server harus hidup pada mesin yang menjalankan monitor; `localhost` pada Codespace lain berbeda.
2. Coba tiap URL dari terminal. API Lab 06 `/health` memeriksa database dan cache; Next `/api/health` hanya liveness Next. Lab 08 juga menyediakan `/api/ready` untuk memeriksa API Lab 06; diskusikan beda target probe.
3. Jalankan `python -B tests/challenge.py` sebelum kelas. Hasil yang diharapkan **9 PASS, 0 FAIL**. Uji ini membuat server HTTP sementara, tidak membutuhkan cloud atau layanan lain.
4. Jangan set `DISCORD_WEBHOOK_URL` untuk demo wajib. Bila memakai webhook opsional, simpan sebagai environment variable lokal/secret, tidak dalam repo atau screenshot.
5. Siapkan tiga terminal server dan satu terminal monitor. Laporan praktik dari mahasiswa masuk ke [hasil/lab10.md](#); lihat [template](#) dan [panduan Git](#).

Susunan kelas

Menit	Demonstrasi dosen	Tugas mahasiswa	Hasil yang dibaca
0-15	Jelaskan target, probe, JSONL, metric/log/trace.	Buka <code>targets.local.json</code> .	URL/port setiap service benar.
15-30	Jalankan tiga layanan dan baseline monitor.	<code>python monitor.py --count 3 --interval 1 --reset</code> .	9 observasi, semua UP.
30-45	Jalankan report/status dan hitung manual.	Bandingkan 3/3, p95.	Kartu UP, availability 100%.
45-60	Hentikan hanya Next Lab 08.	Tambahkan dua probe tanpa <code>--reset</code> .	Next DOWN, dua target lain UP.
60-75	Hitung error budget 99% pada lima check.	Jelaskan 3/5 dan -1,95 check.	Budget negatif berarti terlampaui.
75-85	Hidupkan Next, jelaskan histori dan snapshot.	Probe ulang, ekspor status lagi.	Status terakhir UP, histori gagal tetap ada.
85-100	Tunjukkan log <code>notes_proxy</code> dan checker.	Jalankan 9 check, isi laporan/Git.	Request ID, status, durasi tanpa isi catatan.

Kunci perintah yang ditampilkan di kelas

Baseline: dari root repo Lab 10, `python monitor.py --count 3 --interval 1 --reset`, lalu `python report.py` dan `python status.py --reset` membuka `observations.jsonl` baru; tanpa opsi itu monitor menambahkan baris. Tiga target × tiga putaran = sembilan JSON. Jika semuanya sehat, report menunjukkan `checks=3`, `up=3`, `availability_pct=100.0` untuk tiap nama, sedangkan p95 bervariasi menurut mesin. Buka `status.html` memakai browser atau `Start-Process .\status.html` di Windows.

Gangguan: stop hanya terminal `npm run dev` Lab 08 dengan `Ctrl+C`. Jalankan `python monitor.py --count 2 --interval 1`, `python report.py`, `python status.py`. Hasil uji lokal nyata: Next 3/5 = 60%, `last_ok=false`, `error_budget_total_checks=0.05`, `error_budget_spent_checks=2`, `error_budget_checks=-1.95`; API dan Gradio tetap 5/5, 100%, `last_ok=true`. Hasil latensi yang diukur pada mesin uji berbeda dari mesin kelas.

Pemulihan: jalankan lagi `npm run dev` di repo Lab 08, tambahkan satu putaran monitor tanpa reset. Status terakhir Next menjadi UP, tetapi `availability` kumulatif tetap di bawah 100% karena observasi gagal tidak dihapus. `status.html` baru berubah setelah `python status.py` dieksekusi lagi.

Log: lakukan GET/POST `/api/notes` di Lab 08. Baris JSON server `notes_proxy` memiliki `timestamp`, `level`, `event`, `requestId`, `method`, `status`, `durationMs`. Header `X-Request-ID` pada respons dapat dicocokkan dengan satu baris log. Pada jalur `notes_proxy_unavailable`, log error tidak punya status upstream. Log tidak berisi judul/isi catatan atau token. Metric p95 per target berbeda dari durasi log satu request.

Checker mandiri: `python -B tests/challenge.py` memeriksa config, probe 200/503, penulisan empat observasi, `availability UP/DOWN`, budget negatif, ekspor HTML, escaping nama target, dan penolakan nama duplikat. **9 PASS, 0 FAIL**. Ini menguji perilaku script tanpa menunggu tiga repo; praktik operasional tiga layanan tetap perlu dilakukan terpisah.

Cloud Notes — Status

Dibuat 2026-10-07 12:44 UTC. Data berasal dari synthetic check kelas.

API Compose

UP

Availability sampel: 100.0% (3/3)

p95 respons sukses: 24.12 ms

Sisa budget kegagalan (check): 0.03

Gradio

UP

Availability sampel: 100.0% (3/3)

p95 respons sukses: 7.0 ms

Sisa budget kegagalan (check): 0.03

Next.js

UP

Availability sampel: 100.0% (3/3)

p95 respons sukses: 68.19 ms

Sisa budget kegagalan (check): 0.03

Status tiga layanan saat sehat

Command: `python monitor.py --count 3 --interval 1 --reset`; `python report.py`; `python status.py`, kemudian buka HTML. **Fungsi:** membuat baseline. **Cara kerja:** monitor menyimpan JSONL, report merangkum, `status` menulis snapshot HTML. **Baca hasil:** tiga kartu UP, 3/3.

Cloud Notes — Status

Dibuat 2026-10-07 12:44 UTC. Data berasal dari synthetic check kelas.

API Compose

UP

Availability sampel: 100.0% (5/5)

p95 respons sukses: 97.15 ms

Sisa budget kegagalan (check): 0.05

Gradio

UP

Availability sampel: 100.0% (5/5)

p95 respons sukses: 33.94 ms

Sisa budget kegagalan (check): 0.05

Next.js

DOWN

Availability sampel: 60.0% (3/5)

p95 respons sukses: 68.19 ms

Sisa budget kegagalan (check): -1.95

Status Next DOWN sementara dua target tetap UP

Command: stop Next saja; `python monitor.py --count 2 --interval 1; python report.py; python status.py`. **Fungsi:** mengisolasi gangguan web. **Cara kerja:** dua probe tambahan gagal hanya pada Next, status terakhir dan agregat berubah. **Baca hasil:** Next DOWN 3/5 = 60%, budget -1,95; API/Gradio UP 5/5.

Lab 10 | keluaran perintah yang diuji

\$ python report.py (sesudah Next berhenti)

```
{
  "name": "API Compose",
  "checks": 5,
  "up": 5,
  "availability_pct": 100.0,
  "p95_latency_ms": 97.15,
  "slo_pct": 99.0,
  "error_budget_total_checks": 0.05,
  "error_budget_spent_checks": 0,
  "error_budget_checks": 0.05,
  "budget_exhausted": false,
  "last_ok": true
},
{
  "name": "Gradio",
  "checks": 5,
  "up": 5,
  "availability_pct": 100.0,
  "p95_latency_ms": 33.94,
  "slo_pct": 99.0,
  "error_budget_total_checks": 0.05,
  "error_budget_spent_checks": 0,
  "error_budget_checks": 0.05,
  "budget_exhausted": false,
  "last_ok": true
},
{
  "name": "Next.js",
  "checks": 5,
  "up": 3,
  "availability_pct": 60.0,
  "p95_latency_ms": 68.19,
  "slo_pct": 99.0,
  "error_budget_total_checks": 0.05,
  "error_budget_spent_checks": 2,
  "error_budget_checks": -1.95,
  "budget_exhausted": true,
  "last_ok": false
}
```

Cuplikan output aktual; ditata ulang agar terbaca pada modul.

Keluaran report setelah gangguan

Command: `python report.py`. **Fungsi:** membaca angka SLO langsung dari JSONL. **Cara kerja:** report menghitung per target dan mempertahankan budget negatif bila gagal melampaui jatah. **Baca hasil:** Next 3/5, 60%, -1,95; cuplikan JSON aktual ditata ulang.

Lab 10 | keluaran perintah yang diuji

```
$ python -B tests/challenge.py
```

```
PASS konfigurasi dua target valid
PASS probe sehat menghasilkan HTTP 200
PASS HTTP 503 dihitung DOWN
PASS dua putaran menghasilkan empat observasi
PASS availability UP dan DOWN akurat
PASS budget negatif menunjukkan target melampaui SLO
PASS status web memuat UP dan DOWN
PASS nama target di-escape pada HTML
PASS nama target duplikat ditolak
challenge: 9 PASS, 0 FAIL
```

Cuplikan output aktual; ditata ulang agar terbaca pada modul.

Hasil checker mandiri Lab 10

Command: `python -B tests/challenge.py`. **Fungsi:** memastikan logika monitor sebelum kelas. **Cara kerja:** server HTTP lokal otomatis menyediakan 200 dan 503, lalu script memeriksa laporan/status. **Baca hasil:** 9 PASS, 0 FAIL; keluaran aktual ditata ulang.

Kunci tanya jawab

1. Availability = jumlah check sukses / total $\times$ 100%. Lima check tidak menunjukkan ketersediaan satu bulan, variasi pengguna/lokasi, atau durasi nyata.
2. p95 dihitung dari latensi respons sukses. Perlu jumlah/status error, timeout, interval probe, dan catatan lokasi untuk memahami kegagalan. Kegagalan cepat tetap kegagalan.
3. Metric adalah agregat numerik (availability/p95), log adalah catatan event/request, trace menghubungkan span lintas layanan. Lab ini belum memiliki tracing terdistribusi.
4. Untuk SLO 99% dan 5 check, budget = 0,05 check. Satu kegagalan memberi sisa -0,95; dua memberi -1,95. Pecahan ini ilustrasi skala kecil, bukan kebijakan produksi.

Rubrik dan kesalahan yang sering terjadi

Nilai target/konfigurasi (15%), baseline & JSONL (20%), insiden/isolasi (25%), perhitungan serta interpretasi SLO (25%), log/Git (15%). Jika semua target DOWN, periksa URL, port, dan apakah terminal server ada di host yang sama. Jika report memuat data kelas kemarin, ulangi baseline dengan `--reset` pada file lokal atau gunakan `--output` baru. Jika hanya Gradio DOWN, tunggu startup UI atau cek port 7860. Jika monitor menampilkan target duplikat/error URL, perbaiki `targets.local.json`; jangan menaruh token pada URL. `status.html` adalah file statis, bukan dashboard yang otomatis menyegarkan. Setelah demo, pulihkan Next dan server lain sebelum kelas berikutnya.

Bukti visual tambahan untuk demo

Lab 10 — target dan ringkasan probe — keluaran aktual praktik lokal

\$ Get-Content targets.local.json

```
[
  { "name": "Next.js", "url": "http://127.0.0.1:3000/api/health" },
  { "name": "API Compose", "url": "http://127.0.0.1:8000/health" },
  { "name": "Gradio", "url": "http://127.0.0.1:7860/" }
]
```

\$ python report.py

```
{ "name": "API Compose", "checks": 5, "up": 5, "availability_pct": 100.0, "p95_latency_ms": 33.04, "slo_pct": 99.0,
  "error_budget_checks": 0.050000000000000044, "last_ok": true }
{ "name": "Gradio", "checks": 5, "up": 5, "availability_pct": 100.0, "p95_latency_ms": 3.47, "slo_pct": 99.0,
  "error_budget_checks": 0.050000000000000044, "last_ok": true }
{ "name": "Next.js", "checks": 5, "up": 3, "availability_pct": 60.0, "p95_latency_ms": 35.78, "slo_pct": 99.0,
  "error_budget_checks": 0, "last_ok": false }
```

Target dan ringkasan monitor

Command: buka `targets.local.json`, jalankan `python monitor.py --count 3 --interval 1 --reset` dan `python report.py`. **Fungsi:** menghubungkan URL dengan angka target. **Cara kerja:** nama/URL menjadi kunci pengelompokan JSONL. **Baca hasil:** tiga target dan availability/p95; gambar praktik sebelumnya, angka dapat berbeda.

Lab 10 — log Next.js dari pengujian lokal

GET /api/notes ketika API Lab 06 sengaja dimatikan

```
{ "timestamp": "2026-10-01T00:42:53.548Z", "level": "error", "event": "notes_proxy_unavailable", "requestId": "5cdfe3f3-d8a4-400c-adfa-4779ae3b310b", "method": "GET", "durationMs": 15 }
```

Log memuat level error, event, requestId, dan durasi; tanpa isi catatan.

Log JSON server Next yang aman

Command: `GET /api/notes` saat API Lab 06 mati dan lihat terminal Next. **Fungsi:** membandingkan log per request dengan metric probe. **Cara kerja:** logger mencatat `requestId`, event, dan durasi tanpa isi catatan. **Baca hasil:** `notes_proxy_unavailable`; gambar praktik sebelumnya.